

AERIS Ultimate Operator Cheat Sheet

Full analytical quick-reference for geometry, dataset, aero, sweeps, scripts, artifacts, and operator workflows.

What this guide is for

This is the printable cockpit-reference for the current AERIS software state. It combines the original operator sheet with the completed aero system, including single-case AVL runs, structured result files, sweep execution, sweep inspection, and case-level replay. It is intentionally practical: command-first, option-aware, and workflow-oriented.

Use most often: `aeris geometry generate`, `aeris dataset generate`, `aeris aero run`, `aeris aero sweep`

Use for readback / audit: `aeris dataset inspect`, `aeris aero inspect`, `aeris aero sweep-inspect`, `aeris aero sweep-case-inspect`

Use occasionally: version check, smoke pipeline, visualization scripts, integrity checks, section-count study.

AERIS mental model

- AERIS is a deterministic engineering pipeline, not a bag of scripts.
- Current architecture spine: **geometry -> dataset -> aero -> saved structured result -> sweep -> inspect**.
- Current aero layer is a linear aerodynamic evaluator around operating points, not a flight simulator and not CFD.
- Current next major phase after this sheet: **mass / CG / trim / dynamics foundation**.

A. Command Atlas — What Exists Right Now

Area	Primary commands	Purpose
	<code>aeris version</code> <code>aeris --help</code>	Sanity check, entry-point help
Geometry	<code>aeris geometry generate</code>	Generate one deterministic geometry from config
	<code>aeris dataset generate</code> <code>aeris dataset inspect</code>	Generate many geometries and inspect dataset structure
Pipeline	<code>aeris pipeline smoke</code>	Quick end-to-end sanity check
	<code>aeris aero run</code> <code>aeris aero inspect</code>	Run one aerodynamic case and inspect saved result
Aero (sweep)	<code>aeris aero sweep</code> <code>aeris aero sweep-inspect</code> <code>aeris aero sweep-case-inspect</code>	Run multiple flight conditions for one geometry; inspect whole sweep or one case
	Visualization, architecture checks, dataset audit, section study	Diagnostics, plotting, QC, geometry-resolution selection

B. Core CLI — Geometry / Dataset / Pipeline

B1. General

Use case	Command
	<code>aeris version</code>
Top-level help	<code>aeris --help</code>
	<code>aeris geometry --help</code>
Dataset help	<code>aeris dataset --help</code>
	<code>aeris pipeline --help</code>
Aero help	<code>aeris aero --help</code>

B2. Geometry — primary command

```
aeris geometry generate --config <path>
# short form
aeris geometry generate -c configs/geometry/baseline_bwb.yaml
```

Known/established geometry usage from the reviewed operator material:

Option / pattern	Meaning / notes
	Path to geometry config YAML. This is the main required input shown in the reviewed materials.
Baseline example	<code>aeris geometry generate -c configs/geometry/baseline_bwb.yaml</code>
	<code>aeris geometry generate -c configs/geometry/wing_bwb.yaml</code>

Geometry generation role:

- Loads config and resolves the appropriate generator.
- Samples one design and builds a deterministic geometry realization.
- Writes geometry artifacts and a run manifest under `data/runs/...`

- Acts as the source for later aero runs when the geometry is fresh or reconstructed from stored artifacts.

B3. Dataset — primary commands

<pre>aeris dataset generate --config <path> --n <N> aeris dataset inspect --dataset <dataset_root></pre>	
Option / pattern	Meaning / notes
	Input geometry configuration YAML.
<code>--n</code>	Number of designs to generate.
	Sampler selection, e.g. <code>lhs_v1</code> or <code>random_v1</code> .
<code>--sampler-seed</code>	Seed for modern sampler API.
	Legacy backward-compatible LHS seed style; still works but conceptually legacy.
<code>--name</code>	Custom dataset folder name.
	Force plot saving on or off. Use <code>--no-save-plot</code> for production speed.
<code>--enable-aerosandbox</code> / <code>--no-enable-aerosandbox</code>	Force AeroSandbox build artifacts on or off.
	Inspect existing dataset root and print QC/summary information.

Common dataset examples

<pre># Small LHS dataset aeris dataset generate -c configs/geometry/wing_bwb.yaml --n 20 --sampler lhs_v1 --sampler-seed 123</pre>	
<pre># Fast production dataset (no plots) aeris dataset generate -c configs/geometry/wing_bwb.yaml --n 500 --sampler lhs_v1 --sampler-seed 123 --no-save-plot</pre>	
<pre># AeroSandbox-enabled batch aeris dataset generate -c configs/geometry/wing_bwb.yaml --n 20 --sampler lhs_v1 --sampler-seed 123 --no-save-plot --enable-aerosandbox</pre>	
<pre># Random sampler test aeris dataset generate -c configs/geometry/wing_bwb.yaml --n 20 --sampler random_v1 --sampler-seed 123 --no-save-plot</pre>	
<pre># Inspect a dataset aeris dataset inspect --dataset data/datasets/stress_wing_bwb_n500_lhs123_noplots</pre>	

B4. Pipeline smoke

Use case	Command
	<code>aeris pipeline smoke -c configs/geometry/baseline_bwb.yaml</code>
Smoke test (dev/smoke.toml)	<code>aeris pipeline smoke -c configs/smoke/dev.yaml</code>
	<code>aeris pipeline smoke -c configs/smoke/bad.yaml</code>

C. Aero — Single Case Workflow

C1. What the single aero command does

- Accepts a geometry source (fresh config, existing run directory, or dataset case).
- Builds an AVL-ready geometry view, sets a flight condition, and runs AVL through AeroSandbox.
- Preserves raw artifacts such as `output.txt`, `stability.txt`, `body_derivs.txt`, `strips.txt`, `hinge.txt`, and `avl_stdout.txt`.
- Parses standardized results into core coefficients, stability-axis derivatives, body-axis derivatives, and derived metrics such as `spiral_metric`.
- Runs parser-QC checks against raw AVL-derived values and writes a structured `aero_result.json`.

C2. `aeris aero run` — full option map

Option	Type / default	Purpose
<code>--fresh-geom</code>	Path None	Generate fresh geometry from config for this aero run.
<code>--geom-dir</code>	Path None	Use an existing geometry run directory.
<code>--dataset-root</code>	Path None	Use an existing dataset root.
<code>--geometry-id</code>	str "	Geometry ID inside dataset root; required with <code>--dataset</code> .
<code>--geom-mode</code>	auto	auto native reconstruct. For fresh config runs, auto resolves to native; for stored runs/datasets, auto resolves to reconstruct.
<code>--alpha</code>	float required	Angle of attack [deg].
<code>--beta</code>	float 0.0	Sideslip [deg].
<code>--v-freestream</code>	float 28.0	Flight speed [m/s].
<code>--h-freestream</code>	float 0.0	Altitude [m].
<code>--mach</code>	float 0.0	Mach number slot in the flight condition model.
<code>--roll-rate</code>	float 0.0	Body roll rate [rad/s].
<code>--pitch-rate</code>	float 0.0	Body pitch rate [rad/s].
<code>--yaw-rate</code>	float 0.0	Body yaw rate [rad/s].
<code>--solver-key</code>	aerosandbox_avl	Solver registry key.
<code>--avl-path</code>	str "	Explicit AVL executable path or command, e.g. <code>avl</code> .
<code>--solver-timeout</code>	int 180	Solver timeout.
<code>--spanwise-res</code>	int 4	AVL spanwise paneling resolution.
<code>--chordwise-res</code>	int 8	AVL chordwise paneling resolution.
<code>--spanwise-spacing</code>	equal	AVL spanwise spacing mode.
<code>--chordwise-spacing</code>	cosine	AVL chordwise spacing mode.
<code>--save-surface-forces</code>	bool False	Ask solver to save surface-force output.
<code>--save-element-forces</code>	bool False	Ask solver to save element-force output.

Option	Type / default	Purpose
	int 0	Seed used when fresh geometry is sampled from config.
<code>--geometry-source</code>	str "	Override default run folder name.

Single-run examples

```
# Fresh geometry from config (native source)
aeris aero run --config configs/geometry/baseline_bwb_25.yaml --geometry-source native --alpha 4 --beta 1

# Inspect latest saved run
latest_run="$(ls -td data/runs/*_aero_* | head -n 1)"
aeris aero inspect --run-dir "$latest_run"

# JSON replay
aeris aero inspect --run-dir "$latest_run" --json | python -m json.tool
```

C3. aeris aero inspect

Option	Type / default	Purpose
	Path required	Path to saved aero run root or nested aero/ directory.
<code>--json</code>	bool False	Print machine-readable JSON instead of formatted console output.

D. Aero — Multi-Condition Sweep Workflow

D1. What sweep means in AERIS

- One geometry, many operating points.
- Sweep expansion is deterministic and currently Cartesian-product based over the supplied value lists.
- Each sweep case gets its own folder and its own `aero_result.json`.
- A top-level sweep manifest stores case labels, statuses, flight conditions, and case paths.

D2. `aeris aero sweep` — full option map

Option	Type / default	Purpose
	exclusive sources	Exactly one geometry source must be provided.
<code>--geometry-source</code>	str "	Required with <code>--dataset</code> .
	auto	Native for fresh config, reconstruct for stored geometry unless explicitly overridden.
Base flight condition (roll-rate, pitch-rate, yaw-rate, velocity, altitude, and angle of attack)	floats	Fallback/base values used when a sweep dimension is not explicitly varied.
	CSV string	Comma-separated sweep values for alpha.
	CSV string	Comma-separated sweep values for beta.
	CSV string	Comma-separated sweep values for velocity.
	CSV string	Comma-separated sweep values for altitude.
	CSV string	Comma-separated roll-rate sweep values.
	CSV string	Comma-separated pitch-rate sweep values.
	CSV string	Comma-separated yaw-rate sweep values.
Solver/paneling options	same as single aero run	Same solver, AVL command, timeout, paneling, save flags, seed, and output-name semantics.

Sweep examples

```
# Alpha-q sweep on one geometry
aeris aero sweep --config configs/geometry/baseline_bwb_25.yaml --geometry-source native --velocity 28

# Inspect whole sweep
latest_sweep="$(ls -td data/runs/*_aero_sweep_* | head -n 1)"
aeris aero sweep-inspect --run-dir "$latest_sweep"

# Inspect one sweep case by index
aeris aero sweep-case-inspect --run-dir "$latest_sweep" --case-index 0

# Inspect one sweep case by exact label
aeris aero sweep-case-inspect --run-dir "$latest_sweep" --case-label "case_0000_a+0.000_b+0.000_V28.000_alt"

# JSON case output
aeris aero sweep-case-inspect --run-dir "$latest_sweep" --case-index 0 --json | python -m json.tool
```

D3. Sweep inspection commands

Command	Main options	What it does
	--run-dir, --json	Load saved sweep manifest from run root or nested aero_sweep/; print status breakdown and case list.
aero_sweep_case_resolve	--run-dir, --case-label or --case-index, --json	Resolve a case from saved sweep manifest and replay the case result without rerunning AVL.

E. Result Model, Artifacts, and Saved Files

E1. Structured outputs saved by AERIS aero

Category	Saved / exposed items	Notes
	CL, CD, Cm, CY, Cl, Cn, L/D	Top-level human-facing aerodynamic outputs.
Additional Scalars	CDind, CDff, span efficiency e, Xnp	Promoted to first-class fields because they matter for design and stability interpretation.
	CLa, CLb, CYa, CYb, Cla, Clb, Cma, Cmb, Cna, Cnb, plus rate derivatives and Xnp	Authoritative source for stability-axis derivative family.
Body-axis derivatives	CXu, CXv, CXw, CYu, ..., Cnw	Authoritative source for body-axis derivative family.
	spiral_metric	Computed from the correct derivative family only.
Raw artifacts	output.txt, stability.txt, body_derivs.txt, strips.txt, body.txt, hinge.txt, avl_stdout.txt, airplane.avl, etc.	Audit trail and native solver truth.
	aero_result.json	Machine-readable normalized result.

E2. Single-run folder pattern

```

data/runs/<timestamp>_aero_<label>/
■■■ aero_manifest.json
■■■ <optional copied config>.yaml
■■■ geometry/
■■■ aero/
    ■■■ aero_result.json
    ■■■ airplane.avl
    ■■■ avl_stdout.txt
    ■■■ output.txt
    ■■■ stability.txt
    ■■■ body_derivs.txt
    ■■■ body.txt
    ■■■ hinge.txt
    ■■■ strips.txt
    ■■■ strips_parsed.csv
    ■■■ ...

```

E3. Sweep folder pattern

```

data/runs/<timestamp>_aero_sweep_<label>/
■■■ aero_sweep_manifest.json
■■■ <optional copied config>.yaml
■■■ geometry/
■■■ aero_sweep/
    ■■■ aero_sweep_manifest.json
    ■■■ case_0000_.../
        ■■■ aero_result.json
        ■■■ output.txt
        ■■■ stability.txt
        ■■■ ...
    ■■■ case_0001_.../
        ■■■ ...

```

E4. Trust model

- **Raw AVL files** are the audit/source truth.
- **Parsed AERIS result fields** are the normalized software-facing truth.
- If the two disagree, trust raw files first and assume parser or postprocess drift until proven otherwise.
- Parser-QC was added precisely to catch this class of bug (e.g. the historical spiral metric convention-mixing issue).

F. Useful Scripts — Diagnostics, Visualization, Checks

Use case	Command
	<code>python scripts/bwb_segmented_v1/visualize_airplane.py --mode baseline</code>
Visualize one dataset airplane	<code>python scripts/bwb_segmented_v1/visualize_airplane.py --mode dataset --dataset <path> --geometry-id geom_00001</code>
	<code>python scripts/generic/check_bad_config_failure.py</code>
Architecture integrity check	<code>python scripts/generic/check_architecture_integrity.py</code>
	<code>python scripts/generic/check_metadata_regeneration.py</code>
Seed variation check	<code>python scripts/generic/check_seed_variation.py</code>
	<code>python scripts/generic/check_large_dataset_sanity.py</code>
Audit a dataset	<code>python scripts/generic/audit_dataset.py</code>
	<code>python scripts/bwb_segmented_v1/run_single_case_and_plot_3d.py</code>

Random visualization loop

```

DATASET="data/datasets/<dataset_name>"
ls "$DATASET/geometry" | shuf | head -n 5 | while read GEOM_ID; do
  echo "Visualizing $GEOM_ID"
  python scripts/bwb_segmented_v1/visualize_airplane.py --mode dataset --dataset "$DATASET" --geometry-id "$GEOM_ID"
done

```

G. Section-Count Study — Geometry Resolution Selection

Use this when choosing how many sections / points to use for production geometry generation.

```

python scripts/section_study.py --config configs/geometry/baseline_bwb.yaml --sections 15 25 40 60 100 --points 1000000
python scripts/section_study_pick_default.py --summary-csv data/debug/section_study/baseline_section_study/section_study_summary.csv

```

Resolution tier	Interpretation
	Debug / fast iteration / coarse sanity work
50-45	Standard production range for many studies
	High-resolution reference / convergence studies

H. Very Useful Shell Helpers

```

latest_dataset="$(ls -td data/datasets/* | head -n 1)"
echo "$latest_dataset"
find "$latest_dataset/geometry" -maxdepth 1 -type d | wc -l
wc -l "$latest_dataset/metadata.csv"
wc -l "$latest_dataset/failures.csv"
tail -n 20 "$latest_dataset/logs/dataset.log"

latest_run="$(ls -td data/runs/*_aero_* | head -n 1)"
echo "$latest_run"

latest_sweep="$(ls -td data/runs/*_aero_sweep_* | head -n 1)"
echo "$latest_sweep"

find "$latest_sweep" -maxdepth 3 -type f | sort

```

I. Practical Notes, Operating Rules, and Next-Step Boundaries

- Prefer `--sampler lhs_v1 --sampler-seed 123` over the old `lhs_seed` style unless you are deliberately testing backward compatibility.
- Use `--no-save-plot` for real batch runs. Plotting is a major runtime tax.
- Use **geometry** for one airplane, **dataset** for many airplanes, **aero run** for one operating point, and **aero sweep** for a local multi-condition map around one geometry.
- Current aero workflow is a **linear aerodynamic evaluator around operating points**. It is not a nonlinear flight simulator and not a replacement for CFD.
- Control-surface deflection support was intentionally not added yet. It belongs after the next mass / CG / trim / dynamics foundation, otherwise the numbers are physically under-contextualized.
- The immediate next major subsystem should be **mass properties + CG + trim / dynamics foundation**, not field tensors or NPZ products. Fields/NPZ belong to a later postprocess/data-product phase.
- The directories `data/raw`, `data/interim`, and `data/processed` are mostly later-phase data-lifecycle concerns. Current active operational directories are mainly `data/runs` and `data/datasets`.

J. Recommended operator workflows

Workflow 1 — Generate one geometry

```
aeris geometry generate -c configs/geometry/baseline_bwb.yaml
```

Workflow 2 — Build a dataset

```
aeris dataset generate -c configs/geometry/wing_bwb.yaml --n 100 --sampler lhs_v1 --sampler-seed 123 --no-save-
```

Workflow 3 — Run one aero condition on fresh geometry

```
aeris aero run --config configs/geometry/baseline_bwb_25.yaml --geometry-source native --alpha 4 --beta 0 --vel
```

Workflow 4 — Replay a saved aero result without rerunning AVL

```
latest_run="$(ls -td data/runs/*_aero_* | head -n 1)"
aeris aero inspect --run-dir "$latest_run"
```

Workflow 5 — Sweep alpha and pitch-rate for one geometry

```
aeris aero sweep --config configs/geometry/baseline_bwb_25.yaml --geometry-source native --velocity 28 --altitu
```

Workflow 6 — Inspect one case inside a saved sweep

```
latest_sweep="$(ls -td data/runs/*_aero_sweep_* | head -n 1)"
aeris aero sweep-case-inspect --run-dir "$latest_sweep" --case-index 0
```

Bottom line

AERIS now has a legitimate aerodynamic data engine: deterministic geometry sources, reproducible AVL runs, parsed derivatives, QC-gated structured outputs, multi-condition sweeps, and saved machine-readable results. The next phase is not more command clutter; it is aircraft dynamics awareness through mass, CG, trim, and state-space foundations.